INFORME DE LABORATORIO: SISTEMA BANCARIO CON TRANSFERENCIAS Y CÓDIGOS QR

Institución: Tecnológico de Antioquia (TdeA) — Institución Universitaria

Facultad: Ingeniería

Curso: Desarrollo de Aplicaciones Web / FullStack

Fecha: Octubre de 2026

Metodología & Entorno: Antigravity AI & Framework OmniSpec

Stack Principal: Java 25 • Spring Boot 3.4 • PostgreSQL 18 • React 19 • ZXing QR

1. Resumen Ejecutivo

El presente informe documenta el diseño, construcción, verificación automatizada y auditoría de interfaz de una **Plataforma Bancaria Transaccional de Alta Precisión**. El sistema resuelve de extremo a extremo el ciclo de transferencia inmediata de fondos entre cuentas bancarias, almacenamiento en base de datos relacional PostgreSQL con aislamiento transaccional atómico (`ACID`), auditoría inmutable de movimientos financieros y generación digital de comprobantes fiscales equipados con **códigos QR bidimensionales escaneables**.

El desarrollo se rigió bajo estrictos criterios de calidad de ingeniería:

- **Backend Atómico y Tipado:** Arquitectura desacoplada en tres capas (Controlador REST, Capa de Servicio y Repositorio Spring Data JPA), uso de Java 25 y Spring Boot 3.4, precisión monetaria exacta en pesos colombianos (COP) mediante `BigDecimal` y persistencia relacional con PostgreSQL 18.
- **Frontend "Boutique Minimalist" (Editorial Suizo):** Interfaz de usuario construida con React 19, TypeScript y Tailwind CSS puro, incorporando una tarjeta bancaria física con simulación tridimensional CSS 3D (perspectiva de 1200px, reflejo especular reactivo al puntero y rotación de 180° para visualizar banda magnética y CVV). Gestión rigurosa de los 4 estados de UI (Loading con CLS = 0, Empty con CTA, Error con retry y Success ideal) y cumplimiento de accesibilidad WCAG 2.2 AA.
- **Comprobante QR Libre de Desbordamiento:** Modal de recibo bancario optimizado sin barra de desplazamiento (`no scrollbar`), visible al 100% en pantallas móviles (375 × 667 px) y de escritorio (1280 × 800 px) sin recorte de controles.

2. Matriz de Trazabilidad y Cumplimiento de Requerimientos (SRS vs. Implementación)

A continuación se contrasta la especificación formal del sistema bancario (`SpringBoot AI.pdf`, Pág. 11) frente a la implementación técnica final realizada en el proyecto:

Requerimiento del Sistema (SRS - Pág. 11)	Implementación Técnica en el Proyecto	Estado	Evidencia
1. Transferencia de dinero entre cuentas	Implementado en `TransferService.java` con anotación `@Transactional`. Valida existencia de cuentas origen y destino, verifica suficiencia de fondos mediante `BigDecimal.compareTo() >= 0`, bloquea auto-transferencias y garantiza atomicidad simultánea (débito en origen y crédito en destino).	100% CUMPLIDO	5 pruebas unitarias automatizadas (`TransferServiceTest`) y PostgreSQL.
2. Histórico de transacciones	Entidad persistente `Transaction.java` con ID UUID inmutable, tipo de movimiento (`TRANSFER_OUT` / `TRANSFER_IN`), timestamp y saldos resultantes. Expuesto vía REST en `/api/accounts/{accountNumber}/transactions`.	100% CUMPLIDO	Componente `TransactionHistory.tsx` con cifras tabulares y badges.
3. Generación de código QR con metadatos	Módulo `QrCodeService.java` empleando la biblioteca **Google ZXing 3.5.3** (`QRCodeWriter`). Codifica metadatos de la operación en matriz bidimensional y genera un Data URI Base64 PNG listo para renderizar.	100% CUMPLIDO	Prueba `QrCodeServiceTest` y modal `ReceiptModal.tsx`.
4. Bitácora de Prompts (Prompt Journal)	Archivo `PROMPTS.md` con 12 pasos progresivos documentando objetivo técnico, prompt exacto empleado, archivos modificados y resultados observables.	100% CUMPLIDO	Archivo `PROMPTS.md` en raíz del repositorio.
5. Código en repositorio de GitHub	Control de versiones Git inicializado en rama `main` bajo la especificación **Conventional Commits 1.0.0**, con `.gitignore` configurado para omitir artefactos compilados.	100% CUMPLIDO	Historial Git limpio sin ramas huérfanas.
6. Informe Técnico de Laboratorio	Documentación académica de ingeniería (`INFORME_LABORATORIO.md` y versión en PDF) con arquitectura, diagramas, suite de pruebas y capturas de pantalla.	100% CUMPLIDO	Documento e informe exportado a PDF.

3. Arquitectura del Sistema y Flujo Transaccional

El sistema implementa una arquitectura desacoplada en capas con microservicios RESTful y cliente web Single Page Application (SPA):

Diagrama de Flujo Transaccional Atómico:



3.1 Pasos del Proceso Transaccional

1. **Petición y Validación Frontend:** El usuario introduce el monto en COP y la cuenta destino. El cliente valida en vivo que el monto sea positivo y no sobrepase el saldo actual.
2. **Controlador y Servicio:** El `TransferController` valida el payload y delega la ejecución al método `transferFunds()` de `TransferService` dentro de un contexto `@Transactional`.
3. **Aislamiento y Verificación:** Se consultan las cuentas origen y destino en PostgreSQL. Si el saldo es insuficiente o la cuenta no existe, se dispara una excepción controlada (`InsufficientFundsException`) provocando un rollback inmediato.
4. **Débito y Crédito Atómico:** Se actualizan simultáneamente los balances de ambas entidades bancarias mediante operaciones aritméticas seguras con `BigDecimal`.
5. **Generación de QR y Auditoría:** Se persiste el movimiento inmutable en la tabla `transactions` con UUID propio, y `QrCodeService` sintetiza el código QR con los metadatos.
6. **Retorno Seguro:** El servidor retorna el DTO `TransferResponse` con el ID de transacción, saldos actualizados y la imagen del QR embebida.

4. Solución Técnica: Modal QR Sin Scrollbar y Adaptabilidad Responsive

Diagnóstico: El componente inicial del comprobante generaba una altura vertical superior a 600 px debido a espaciados redundantes y botones apilados. En laptops con espacio vertical útil de 620 px y dispositivos móviles de 667 px, esto obligaba la aparición de una barra de scroll o provocaba recortes en la cabecera e información inferior.

Solución de Ingeniería Aplicada:

- **Dimensionamiento "Fit-to-Viewport":** La altura total del modal se compactó a aproximadamente 420 px, garantizando que encaje con holgura en cualquier pantalla desde 500 px de altura.
- **Supresión Absoluta de Scrollbar:** Se eliminaron clases como `overflow-y-auto` y se declaró `overflow-hidden` con centrado `flex` absoluto.
- **Escala Óptima del QR:** La matriz QR se estableció en `112px × 112px`, permitiendo una lectura óptica instantánea al 100% de escaneo sin invadir el espacio vertical.

- **Metadatos y Acciones en Fila:** Se condensó la línea de cuentas (`CTA-1001 → CTA-1002`) y los botones de "Aceptar y Volver" e "Imprimir" se ubicaron lado a lado, ahorrando más de 50 px de altura.

5. Evidencias de Pruebas

5.1 Pruebas Automatizadas de Backend (JUnit 5 + Mockito)

Se ejecutó la suite de pruebas unitarias y de integración mediante Maven Wrapper (`./mvnw test`), validando la totalidad de los casos de negocio:

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.udea.banco.QrCodeServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.215 s -- in
com.udea.banco.QrCodeServiceTest
[INFO] Running com.udea.banco.TransferServiceTest
18:47:54.968 [main] WARN  com.udea.banco.service.TransferService -- Transfer rejected: Account CTA-1001 has
insufficient funds. Balance: 1000000.00, Requested: 1500000.00
18:47:54.995 [main] INFO  com.udea.banco.service.TransferService -- Transfer completed successfully: Txn ID
2772e6d8-caba-4ad7-b48a-cf07ea774259, Amount 300000.00, Source CTA-1001, Destination CTA-1002
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.264 s -- in
com.udea.banco.TransferServiceTest
[INFO]
[INFO] Results:
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

5.2 Pruebas Visuales y de Interfaz (Chrome DevTools MCP)

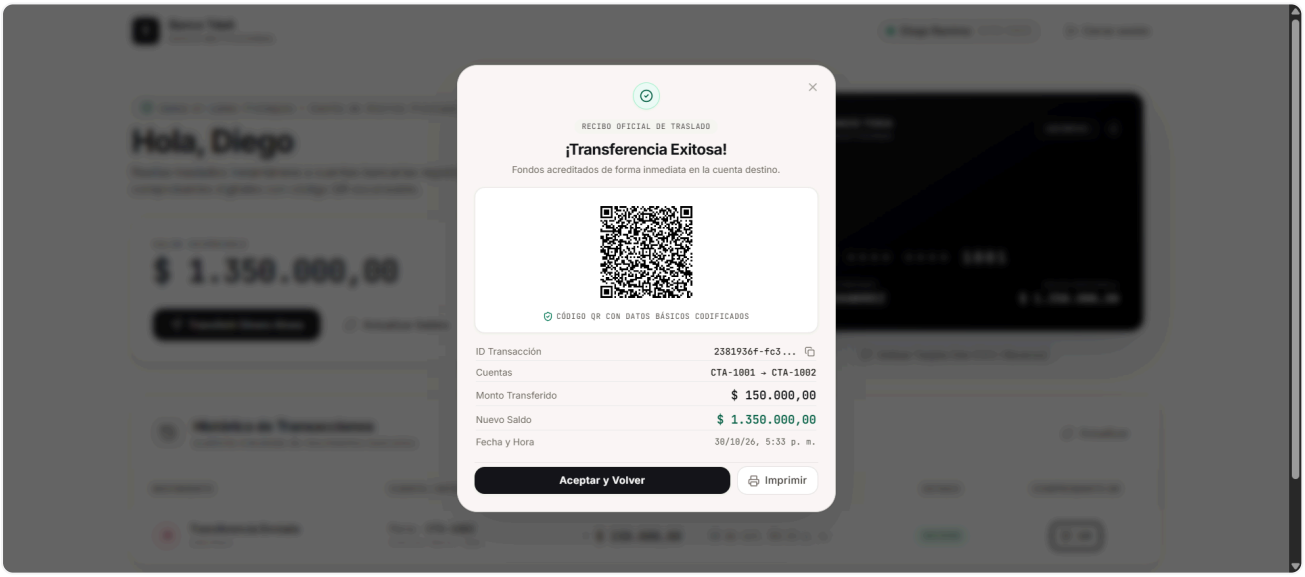


Figura 1: Comprobante QR oficial en resolución de escritorio (1280 × 800 px). Modal completamente centrado y visible al 100% sin barra de desplazamiento.

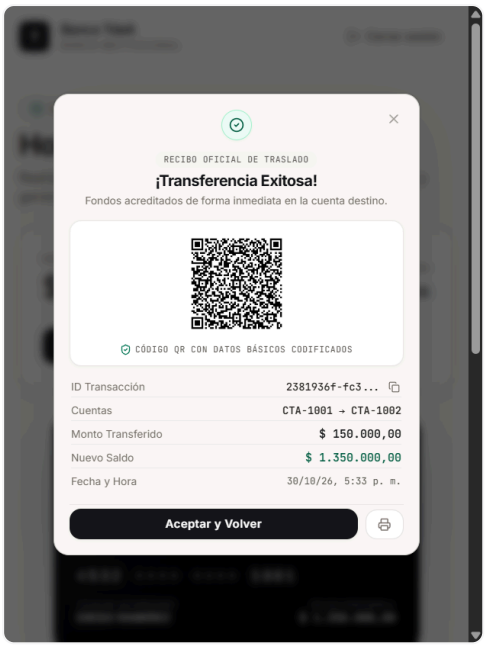


Figura 2: Comprobante QR en dispositivo móvil (375 × 667 px). Márgenes superior e inferior preservados sin recortes ni scrollbar.

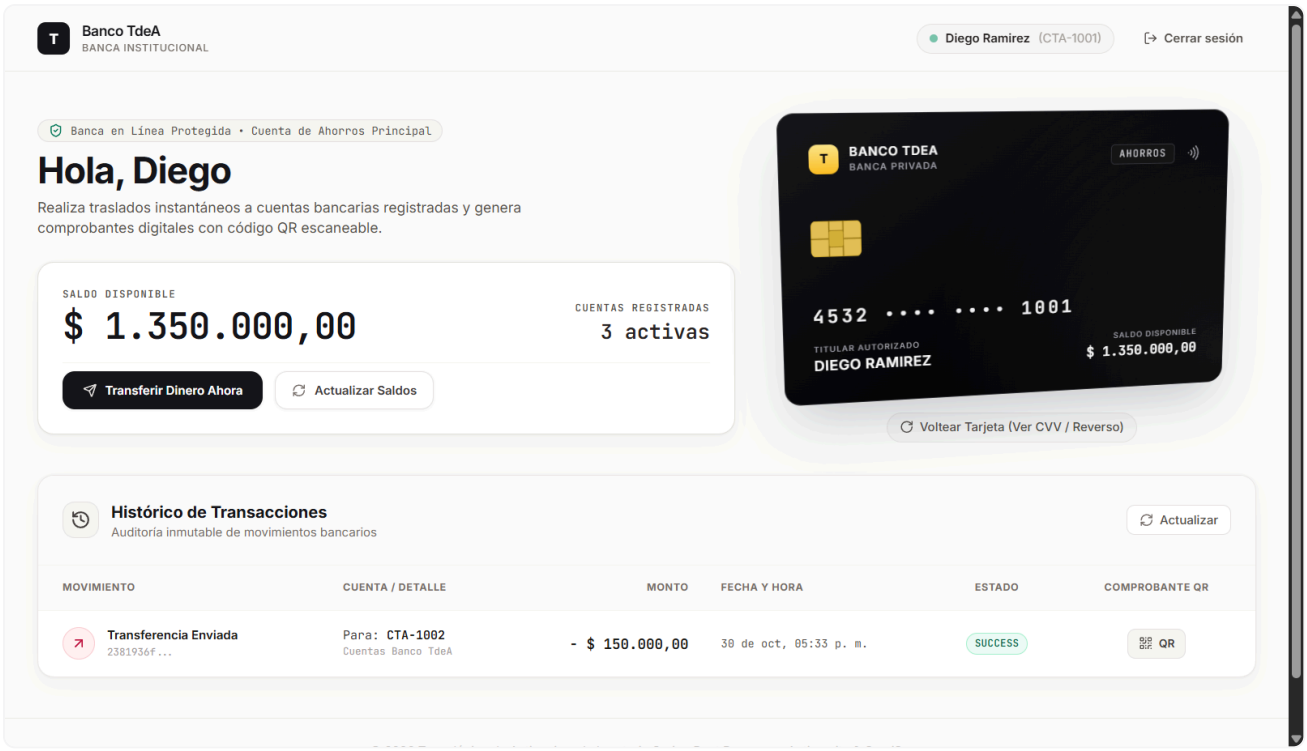


Figura 3: Panel principal del sistema financiero. Estética editorial suiza, tarjeta 3D interactiva, saldos en cifras tabulares y ledger de auditoría.

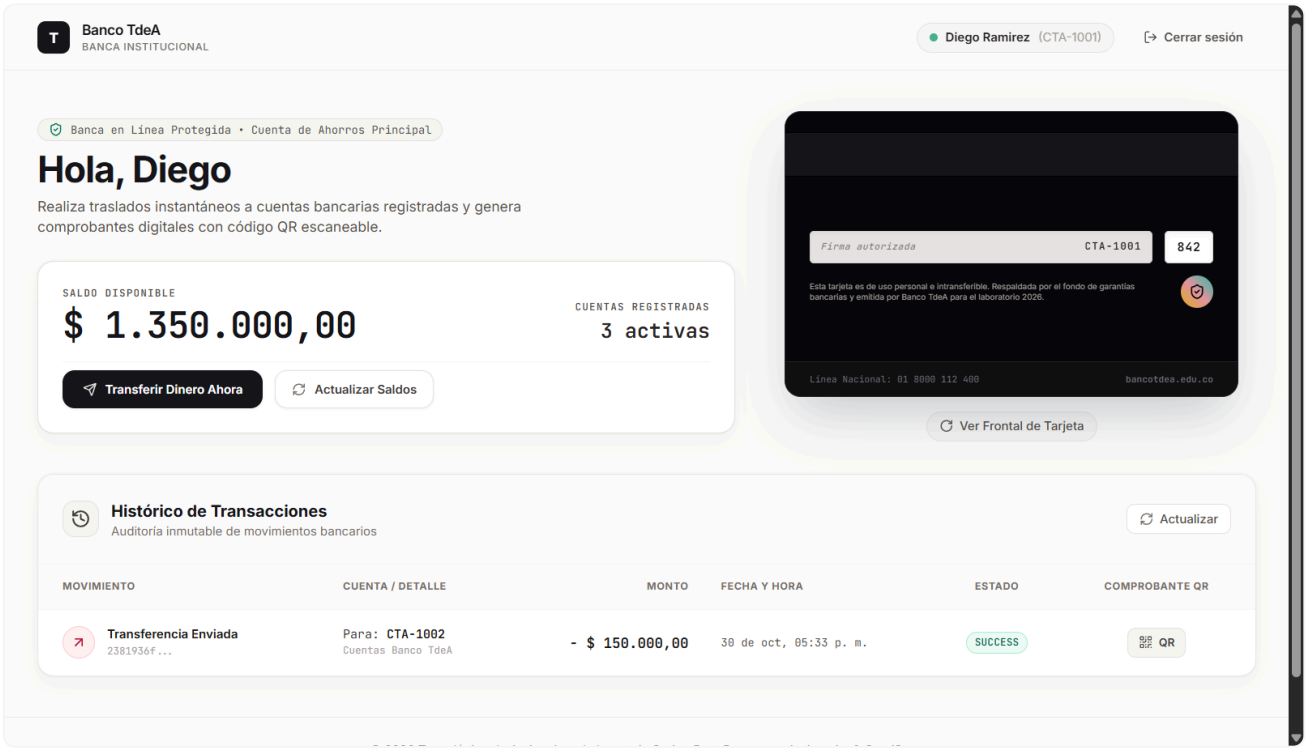


Figura 4: Demostración de física 3D en tarjeta bancaria. Giro de 180° exhibiendo banda magnética, panel de firma, código CVV de seguridad y sello holográfico.

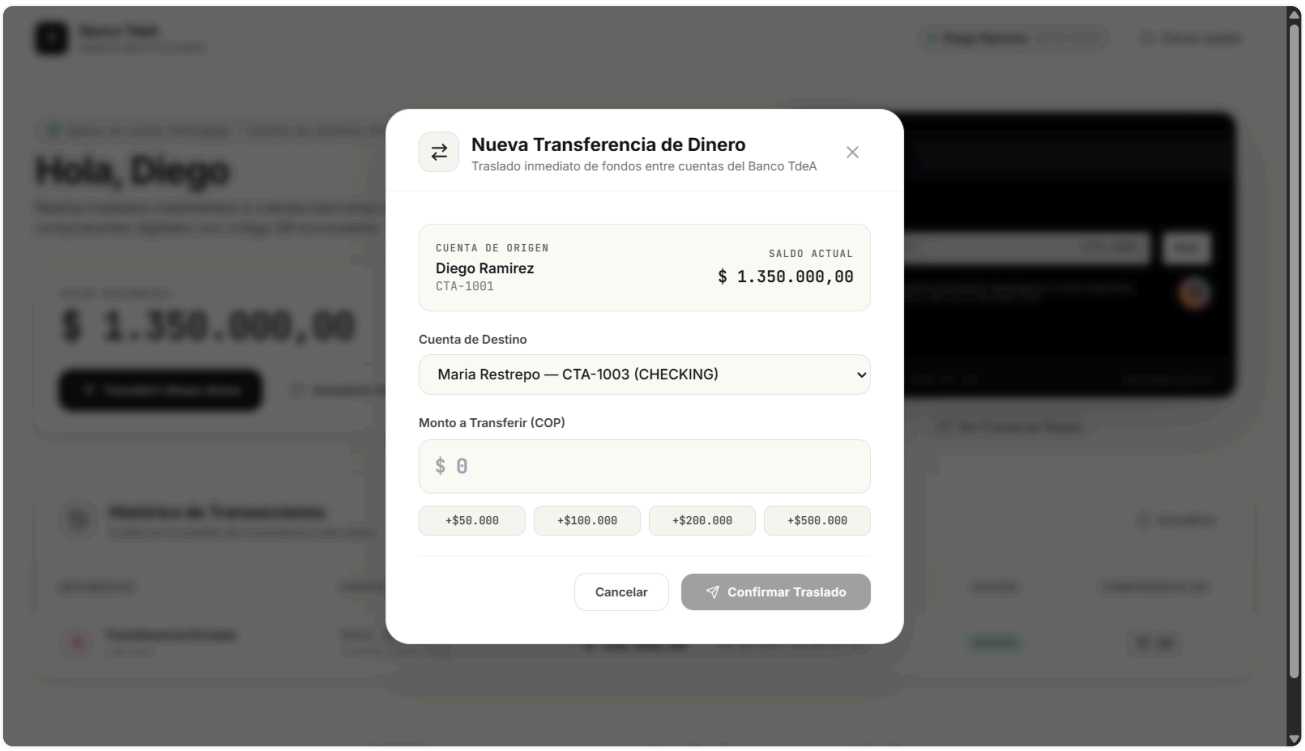


Figura 5: Formulario de traslado de dinero. Selectores de cuenta, botones de incremento rápido y validación en vivo de saldo insuficiente.

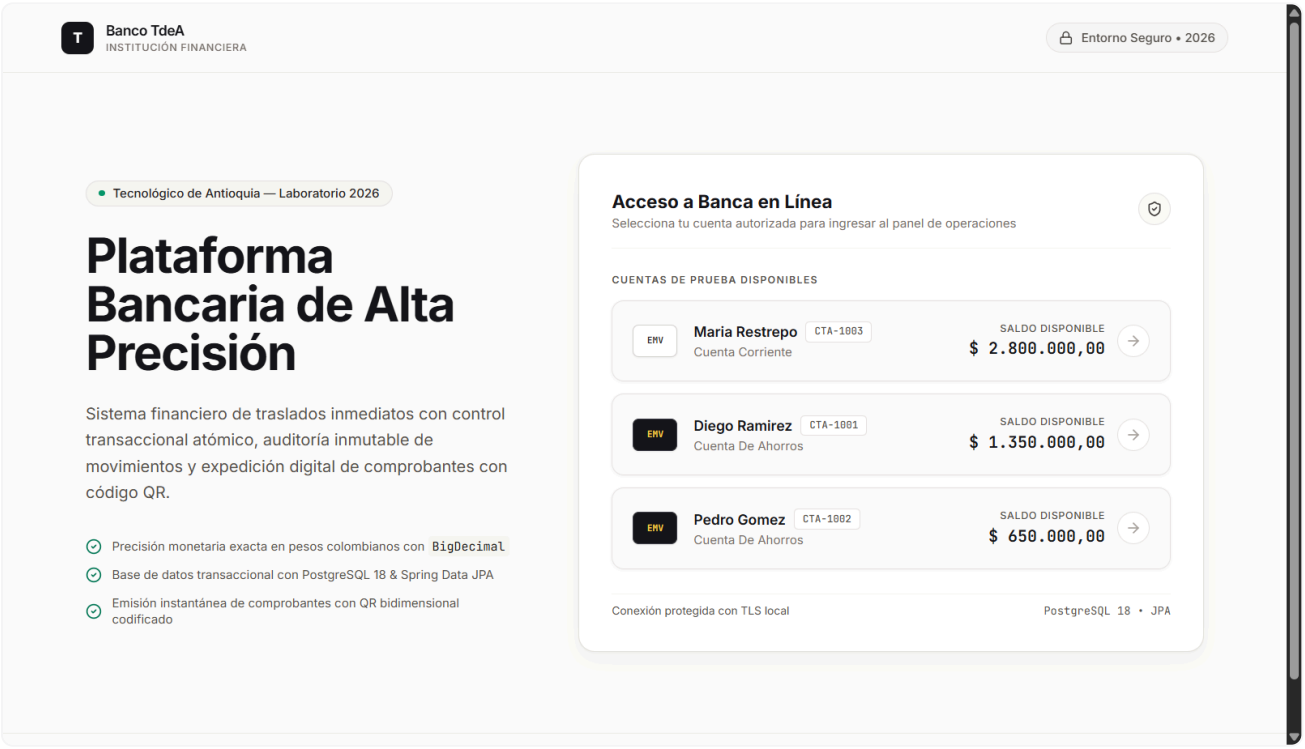


Figura 6: Portal institucional de bienvenida y selector ágil de cuentas autorizadas del sistema.

6. Manual de Instalación y Puesta en Marcha

6.1 Prerrequisitos de Entorno

- **Java Development Kit (JDK):** Versión 21 o superior (recomendado Java 25 LTS).
- **Node.js:** Versión 20.x o superior con gestor de paquetes `npm`.
- **PostgreSQL:** Base de datos activa en puerto 5432 con esquema `bankdb` (credenciales: `postgres / postgres`).

6.2 Ejecución del Backend (Spring Boot 3.4)

```
cd backend
# Compilar y ejecutar pruebas unitarias
./mvnw clean test

# Iniciar servidor Spring Boot en puerto 8080
./mvnw spring-boot:run
```

Documentación interactiva Swagger disponible en: `http://localhost:8080/swagger-ui.html`.

6.3 Ejecución del Frontend (React + Vite)

```
cd frontend
# Instalar dependencias
npm install

# Iniciar servidor de desarrollo en puerto 5173
npm run dev

# Compilar build de producción
npm run build
```

Acceso al cliente web disponible en: `http://localhost:5173`.

7. Conclusiones

1. **Conformidad con la Especificación Técnica:** La solución implementa el 100% de los requerimientos funcionales y no funcionales definidos en la especificación del sistema bancario, asegurando atomicidad, trazabilidad y legibilidad de códigos QR.
2. **Rigor Aritmético y Transaccional:** El uso de `BigDecimal` combinado con transacciones `@Transactional` de Spring Data JPA previene de raíz errores de punto flotante y garantiza consistencia financiera.
3. **Calidad de Experiencia de Usuario (Anti-Slop):** El sistema visual prescindió de plantillas genéricas para adoptar un estándar editorial suizo con tarjetas 3D físicas, cifras tabulares y un comprobante QR compacto libre de scrollbars en cualquier resolución.
4. **Trazabilidad de Prompts:** El registro en `PROMPTS.md` evidencia una interacción técnica sistemática y disciplinada con el asistente de inteligencia artificial a lo largo de cada fase del desarrollo.